Assignment A3 : Discriminative Fine-tuning

Individual assignment • 20% individual-report component Released: 3 Dec 2026 Due: 10 Dec 2026, 23:59

Objective. Climb a three-rung ladder on the same task — bag-of-words, pooled embeddings, then a fine-tuned transformer — and quantify what each rung buys. Learn the HuggingFace training loop, and learn *when a bigger model is worth it.*

From the lecture

The hands-on counterpart to **Discriminative Task** (3 Dec AM, Anand) and **Neural LM & Contextual Embeddings** (3 Dec PM, Yang) — bag-of-words → embeddings → attention, exactly the lecture arc. **Pre-watch:** video #04 Word Embeddings, #05 Attention, #06 Evaluating Classifiers, #07 Contrastive Learning.

Data — a public Kaggle competition. All assignments build on **Jigsaw Unintended Bias in Toxicity Classification** (`jigsaw-unintended-bias-in-toxicity-classification`). Join the competition on Kaggle, then fetch it with the Kaggle API or via `starter/load_data.py` (which uses `kagglehub`). From its `train.csv` we derive, through the starter helpers:

- **Classification** — binary toxicity (`target` ≥ 0.5 = toxic) from `comment_text`, with the **identity columns** (`male`, `female`, `black`, `white`, `christian`, `muslim`, ...) as the **subgroups** for fairness analysis. Helper: `load_textclf()` → train / validation / test splits.
- **Raw text for LM work** — a fixed sample of `comment_text` with a held-out split. Helper: `load_lm()` → (train, held-out) lines.

Models come from HuggingFace, as usual. `check_submission.py` validates your output files before you submit; the grading labels for the `test` split are held out by the teaching team. *Content note: the comments can contain offensive language.*

Scope

Required core: the bag-of-words baseline and the DistilBERT fine-tune (rungs 1 and 3), with their two prediction files — this is the comparison that carries the lesson.

Optional: the pooled-embedding middle rung (task 2) and its prediction file. It nicely shows the in-between, but the BoW→BERT contrast stands without it.

1. Tasks

Using `load_textclf()`: train on the `train` split, tune on `validation`, and predict the `test` split with each of:

1. **Bag-of-words logistic regression** — `TfidfVectorizer` + `LogisticRegression` (scikit-learn). This is your baseline.
2. **[optional] Pooled-embedding logistic regression** — mean-pool a frozen sentence embedding (`sentence-transformers`, e.g. `all-MiniLM-L6-v2`) and fit logistic regression on top.
3. **Small transformer fine-tune** — fine-tune `distilbert-base-uncased` on the task (2–3 epochs, `max_len`=128).

Submit predictions for the two required models (and the embedding model if you do the optional rung).

Runs on your laptop

The provided train split is sized so DistilBERT fine-tunes in **under ∼30 min** on a modern laptop (Apple-Silicon `mps` or CPU; set `fp16` off on CPU). Rungs 1–2 take seconds. If your machine is slow, a free Colab T4 is an *optional* fallback — but it is not required. Keep epochs/`max_len` as specified so runtime stays bounded.

Deliverable & auto-grading

Submit prediction files, each with columns `id,pred`: `a3_pred_bow.csv`, `a3_pred_bert.csv` (required), and `a3_pred_emb.csv` ([optional]). Grading reports macro-F1 of each against the hidden test reference. **Pass condition:** the DistilBERT model must beat the bag-of-words baseline by a clear margin (threshold on Brightspace); the embedding model is expected to land in between.

Notes. The label is binary toxicity (`toxic`/`nontoxic`); use the `validation` split for model selection (not the hidden test). The tiny-config caps above (`max_len`=128, 2–3 epochs) are **mandatory** — they are what keep the fine-tune laptop-sized. "BERT beats BoW" is expected here because the task is built to reward word order and context that a bag of words cannot see.

2. Findings prompt

Where did embeddings help, and what did fine-tuning fix that embeddings could not? Point to concrete examples that rung 2 got wrong and rung 3 got right.

3. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**
3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them.* Using AI is allowed; undisclosed or unverified use is not.

4. Submission

Upload a single `zip` to Brightspace named `A3_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.